

LAB REPORT 13

Structures in C++

Computer Programming · UET Nowshera · Electrical Engineering

1 Objective

The aim of this lab was to learn how C++ lets a programmer bundle several related pieces of data — regardless of their types — into a single named package called a structure. The session worked toward the following outcomes:

- Appreciating why grouping mixed-type data under one name is more organised than using scattered separate variables
- Writing a `struct` definition and understanding the role of the semicolon that closes it
- Instantiating structure variables and reading or writing their fields through the dot operator
- Handling more than one structure variable at a time to represent multiple real-world records

2 Theoretical Background

The Need for Structures

An array is a tidy way to store many values of the *same* type, but it falls short the moment the data involves mixed types. A student record, for instance, needs text for the name, a whole number for the age, and a character for the grade. A **structure** solves this by acting as a custom container whose members can each be of a completely different type, yet all belong to the same named variable.

Writing a Structure Definition

The keyword `struct` opens the definition, followed by the tag name chosen by the programmer and a pair of curly braces that list the member declarations. The closing brace *must* carry a semicolon — a point beginners frequently miss. The definition itself commits no memory; it is purely a blueprint. Storage is only reserved when the programmer declares an actual variable of that structure type.

The Dot Operator for Member Access

Once a structure variable exists, individual fields are reached by placing the **dot operator** (.) between the variable name and the member name. The syntax reads naturally: `p1.age = 20;` writes 20 into the age field of `p1`, and `cout << p1.age` reads it back out. The same operator works for every member regardless of its type.

Copying Structures and Nesting

Assigning one structure variable to another of the same type with a plain = copies every member in one shot — no need to transfer fields individually. Structures can also serve as members of other structures, which makes modelling layered real-world data (such as an address embedded inside a person) entirely natural.

3 Software Used

IDE	DEV-C++ IDE
COMPILER	C++ Compiler (bundled with DEV-C++)

4 Experimental Tasks

Task 1 — Structure Declaration: Person

A Person structure was put together with three fields: name, age, and address. One variable of this type was brought to life, its fields were filled in using the dot operator, and the complete record was printed to the console.

● ● ● C++ Code

```
#include <iostream>
#include <string>
using namespace std;

struct Person {
    string name;
    int    age;
    string address;
};

int main() {
    Person p1;
    p1.name   = "Ahmed Khan";
    p1.age    = 20;
    p1.address = "Nowshera, KPK";

    cout << "Name:    " << p1.name    << endl;
    cout << "Age:      " << p1.age     << endl;
    cout << "Address: " << p1.address << endl;
    return 0;
}
```

Task 2 — Structure Declaration: Student

A Student structure was defined to hold a name, an integer age, and a single character grade. A variable called student1 was created, populated field by field, and its contents were printed to verify the assignments worked as expected.

● ● ● C++ Code

```
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string name;
    int    age;
    char   grade;
};

int main() {
    Student student1;
    student1.name = "Sara Bibi";
    student1.age  = 19;
    student1.grade = 'A';

    cout << "Name:  " << student1.name << endl;
    cout << "Age:   " << student1.age  << endl;
    cout << "Grade: " << student1.grade << endl;
    return 0;
}
```

Task 3 — Multiple Structure Variables: Book

A Book structure was built with fields for title, author, price, and page count. Two separate variables of this type were created and loaded with different data, then both were displayed to show how a single structure definition can serve as the template for an unlimited number of independent records.

● ● ● C++ Code

```
#include <iostream>
#include <string>
using namespace std;

struct Book {
    string title;
    string author;
    float price;
    int pages;
};

int main() {
    Book book1 = {"C++ Primer", "Lippman", 49.99f, 976};
    Book book2 = {"Clean Code", "Martin", 35.00f, 464};

    cout << "--- Book 1 ---" << endl;
    cout << "Title: " << book1.title << " Author: " << book1.author << endl;
    cout << "Price: " << book1.price << " Pages: " << book1.pages << endl;

    cout << "--- Book 2 ---" << endl;
    cout << "Title: " << book2.title << " Author: " << book2.author << endl;
    cout << "Price: " << book2.price << " Pages: " << book2.pages << endl;
    return 0;
}
```

5 Conclusion

Structures turned out to be a natural fit for the kinds of data that do not fit neatly into a single array. By building Person, Student, and Book structures, the tasks showed how a struct definition creates a reusable mould that any number of variables can be stamped from. The dot operator gave clean, readable

access to each field, and the single-statement copy behaviour was a pleasant contrast to copying array elements one at a time. The pattern of grouping related data into a named type is the same principle that underpins classes and object-oriented design.